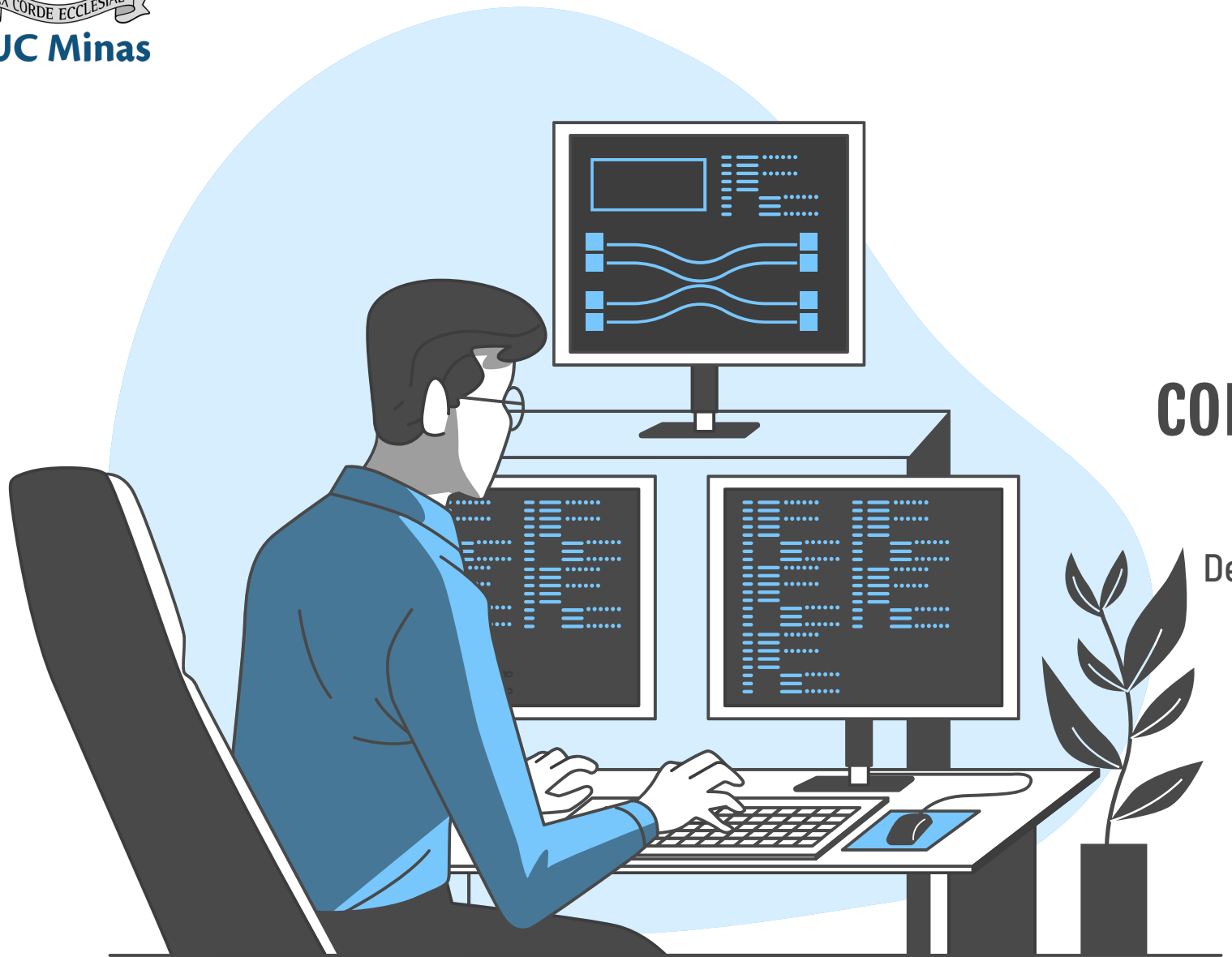
PUC Minas



Aula 06 · FullStack

Rodando tudo junto: um comando para front e back

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Chamar o Maven Wrapper a partir de um script npm
2. Usar o **concurrently** para rodar dois processos com saída colorida
3. Lidar com as diferenças entre **Windows** e Linux/macOS
4. Configurar o **proxy** do Vite (`server.proxy`)
5. Escolher entre CORS e proxy em cada ambiente

Antes de começar

- Aula 04 desta pasta

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 O incômodo

02 Passo 1: script que sobe o back

03 Passo 2: concurrently

04 Alternativa: proxy do Vite

05 Bônus: tarefas do VS Code

06 Aprofundando: produtividade no desenvolvimento

01

O incômodo

O incômodo

Até aqui você abria dois terminais, entrava em `Back` para rodar o Maven e em `Front` para rodar o Vite. O projeto `Exemplo_Integracao` resolve isso com scripts no `package.json` do Front.

01

O incômodo

Aula 06 · FullStack

02

Passo 1: script que sobe o back

Passo 1: script que sobe o back

Front/package.json (trecho)

```
"scripts": {
  "dev": "vite",
  "back": "cd ../Back && ./mvnw spring-boot:run",
  "back:windows": "cd ..\\Back && mvnw.cmd spring-boot:run"
}
```

Passo 1: script que sobe o back

Com isso, de dentro de `Front/`, `npm run back` sobe o Spring Boot. O caminho `../Back` funciona porque as duas pastas estão lado a lado (aula 02).

02

Passo 1: script que sobe o back

Aula 06 · FullStack

03

Passo 2: concurrently

Passo 2: concurrently

O **concurrently** roda vários comandos ao mesmo tempo no mesmo terminal, com um prefixo colorido em cada linha. `Ctrl+C` encerra todos.

Instalação (dentro de Front/)

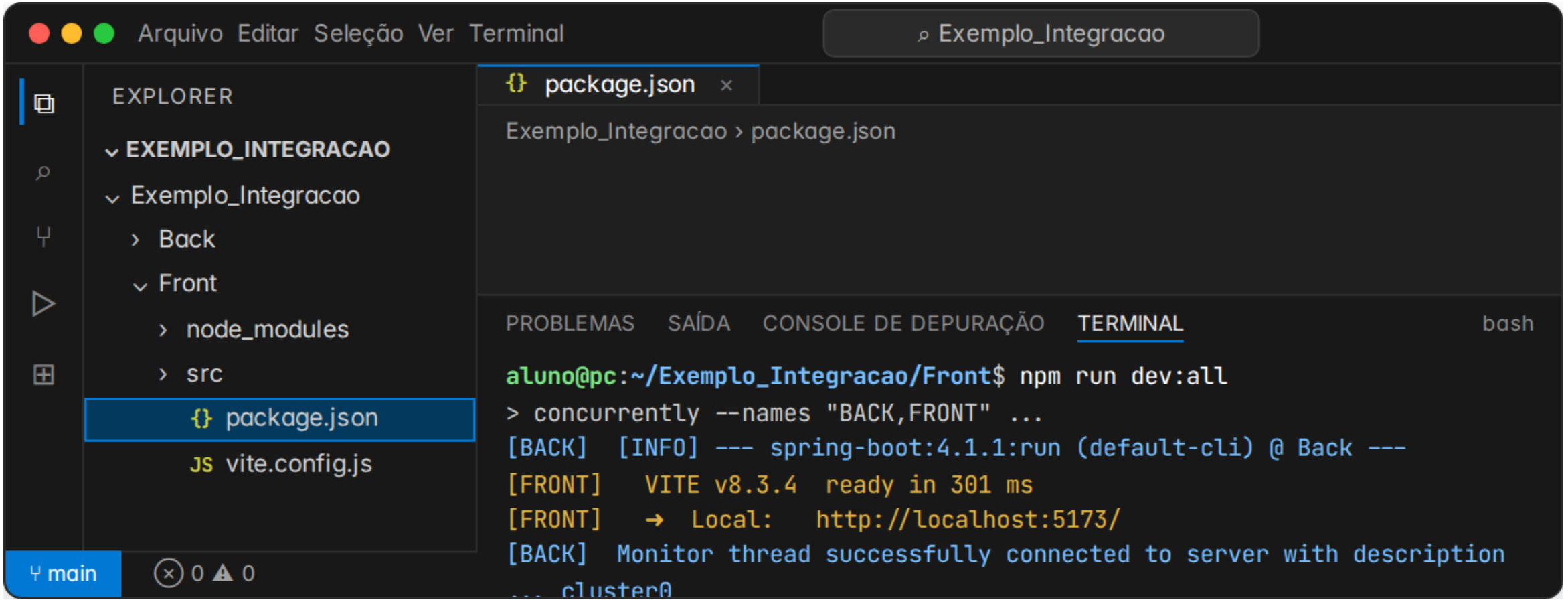
```
npm install -D concurrently
```

Passo 2: concurrently

Front/package.json (igual ao Exemplo_Integracao)

```
"scripts": {
  "dev": "vite",
  "back": "cd ../Back && ./mvnw spring-boot:run",
  "back:windows": "cd ..\\Back && mvnw.cmd spring-boot:run",
  "dev:all": "concurrently --names \"BACK,FRONT\" --prefix-colors \"blue,magenta\" \"npm run back\" \"npm run dev\"",
  "dev:all:windows": "concurrently --names \"BACK,FRONT\" --prefix-colors \"blue,magenta\" \"npm run back:windows\" \"npm run
dev\""
}
```

Passo 2: concurrently



Um único terminal com os dois servidores. As linhas azuis são do Spring, as rosas do Vite.

Passo 2: concurrently

Observação

O Vite fica pronto em 1 segundo e o Spring em alguns segundos. Nesse intervalo o front mostra “back-end offline”. Por isso o `App.jsx` do projeto tenta o `/api/status` de novo a cada 5 segundos (aula 04): quando o Spring termina de subir, o painel fica verde sozinho.

Passo 2: concurrently

Erro comum

`'.' não é reconhecido como um comando` no Windows: use `npm run dev:all:windows` (que chama `mvnw.cmd`). `./mvnw: Permission denied` no Linux/macOS: `chmod +x ../Back/mvnw`. `JAVA_HOME is not set`: o terminal do Node não está enxergando o JDK (Back-end, aula 01).

04

Alternativa: proxy do Vite

Alternativa: proxy do Vite

O servidor de desenvolvimento do Vite pode **repassar** chamadas para o Spring. O navegador fala só com `localhost:5173`, e o Vite, por trás, fala com `localhost:8080`. Como para o navegador é a **mesma origem**, não existe CORS.



Com proxy, o navegador nunca chama a porta 8080 diretamente.

Alternativa: proxy do Vite

vite.config.js

```
export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    strictPort: true,
    proxy: {
      "/api": {
        target: "http://localhost:8080",
        changeOrigin: true,
      },
    },
  },
});
```

config/apiConfig.js (versão com proxy)

```
// Em dev, URL relativa: o proxy do Vite
// encaminha /api para o Spring.
// Em produção, VITE_API_URL aponta
// para o Render.
const API_CONFIG = {
  BASE_URL: (import.meta.env.VITE_API_URL
    ?? "").replace(/\/+$/, ""),
};
export default API_CONFIG;
```

Alternativa: proxy do Vite

	CORS no Spring	Proxy do Vite
Desenvolvimento	Front chama `localhost:8080` direto	Front chama `/api`, Vite repassa
Produção (Vercel + Render)	**Continua necessário**	**Não existe** proxy: o Vercel só serve arquivos. Precisa de CORS do mesmo jeito.
Configuração	`CorsConfig` + `app.cors.allowed-origins`	`server.proxy`
Indicado para	Os projetos da disciplina (mesmo código local e em produção)	Esconder o CORS em dev, ou quando o Spring serve o front (aula 07)

Alternativa: proxy do Vite

Atenção

O proxy só existe no `npm run dev`. Se o seu front só funciona por causa dele, ao publicar no Vercel aparecerá o erro de CORS que estava escondido. Por isso os projetos da disciplina preferem configurar o CORS desde o início.

05

Bônus: tarefas do VS Code

Bônus: tarefas do VS Code

Para subir tudo pelo menu **Terminal → Executar Tarefa**, crie `.vscode/tasks.json` na raiz:

.vscode/tasks.json

```
{
  "version": "2.0.0",
  "tasks": [
    { "label": "Back",  "type": "shell", "command": "./mvnw spring-boot:run",
      "options": { "cwd": "${workspaceFolder}/Back" }, "isBackground": true },
    { "label": "Front", "type": "shell", "command": "npm run dev",
      "options": { "cwd": "${workspaceFolder}/Front" }, "isBackground": true },
    { "label": "Full stack", "dependsOn": ["Back", "Front"] }
  ]
}
```

06

Aprofundando: produtividade no desenvolvimento

Aprofundando: produtividade no desenvolvimento

Opção do concurrently	Efeito
`--names "BACK,FRONT"`	Prefixo de cada linha
`--prefix-colors "blue,magenta"`	Cores dos prefixos
`--kill-others`	Se um processo cair, encerra o outro
`--success first`	Código de saída igual ao do primeiro processo
`--raw`	Saída sem prefixos

Aprofundando: produtividade no desenvolvimento

Scripts na raiz: npm run dev sobe tudo de qualquer lugar

```
// package.json (raiz do monorepo, opcional)
{
  "private": true,
  "scripts": {
    "dev": "concurrently -n BACK,FRONT -c blue,magenta \"npm:back\" \"npm:front\"",
    "back": "cd Back && ./mvnw spring-boot:run",
    "front": "npm --prefix Front run dev",
    "test": "cd Back && ./mvnw test && cd ../Front && npm run test:run"
  },
  "devDependencies": { "concurrently": "^9.2.4" }
}
```

Proxy do Vite x CORS: decisão

Use proxy quando

- O front será servido pelo próprio Spring (aula 07)
- Você quer cookies sem configuração cross-site
- A API não precisa ser chamada de outros sites

vs

Use CORS quando

- Front no Vercel e back no Render
- Outros clientes (app, parceiros) chamam a API
- Você quer o mesmo código em dev e produção

Aprofundando: produtividade no desenvolvimento

Mais opções do proxy

```
proxy: {
  "/api": { target: "http://localhost:8080", changeOrigin: true },
  "/ws":  { target: "ws://localhost:8080", ws: true },              // WebSocket
  "/legado": {
    target: "http://localhost:8080",
    rewrite: (caminho) => caminho.replace(/^\/legado/, ""),          // remove o prefixo
  },
}
```

Aprofundando: produtividade no desenvolvimento

Dica

No VS Code, use o **Spring Boot Dashboard** para iniciar/parar o back com um clique e o terminal só para o front. É uma alternativa visual ao `concurrently`, útil para depurar o back com breakpoints enquanto o front roda.

Aprofundando: produtividade no desenvolvimento

1. Instale `concurrently` como devDependency no Front.
2. Adicione os scripts `back`, `back:windows`, `dev:all` e `dev:all:windows`.
3. Rode `npm run dev:all` (ou a versão Windows).
4. Espere o log do Spring mostrar *Started* e confira o painel de status do front.
5. `Ctrl+C` encerra os dois processos.

07

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

O que o `concurrently` faz?

A Compila o Java

B Roda vários comandos ao mesmo tempo no mesmo terminal

C Faz deploy

D Gera testes

Quiz · questão 1 de 5

Resposta

O que o `concurrently` faz?

A Compila o Java

✓ Roda vários comandos ao mesmo tempo no mesmo terminal

C Faz deploy

D Gera testes

Por quê: Ctrl+C encerra todos.

Pergunta

No Windows, qual script sobe o back no projeto Exemplo_Integracao?

A npm run back

B npm run back:windows

C npm start

D mvn run

Resposta

No Windows, qual script sobe o back no projeto Exemplo_Integracao?

A npm run back

✓ npm run back:windows

C npm start

D mvn run

Por quê: Ele usa mvnw.cmd.

Pergunta

O proxy do Vite funciona em produção no Vercel?

A Sim

B Não, só existe no npm run dev

C Só com HTTPS

D Só no Render

Resposta

O proxy do Vite funciona em produção no Vercel?

A. Sim

✓ Não, só existe no npm run dev

C. Só com HTTPS

D. Só no Render

Por quê: Em produção é preciso CORS.

Pergunta

Com proxy, a BASE_URL em desenvolvimento fica:

A http://localhost:8080

B Vazia (URLs relativas como /api)

C https://render.com

D localhost:5173/8080

Resposta

Com proxy, a BASE_URL em desenvolvimento fica:

A http://localhost:8080

✓ Vazia (URLs relativas como /api)

C https://render.com

D localhost:5173/8080

Por quê: O Vite repassa /api para o Spring.

Pergunta

Por que o front deve tolerar o back que demora a subir?

A Estética

B O Vite sobe em 1 s; o Spring leva vários segundos

C Por causa do CORS

D Não deve

Resposta

Por que o front deve tolerar o back que demora a subir?

A Estética

✓ O Vite sobe em 1 s; o Spring leva vários segundos

C Por causa do CORS

D Não deve

Por quê: Verifique o status periodicamente.

Fechamento

Resumo da aula

- ✓ Scripts npm podem chamar o Maven: `cd ../Back && ./mvnw spring-boot:run`.
- ✓ `concurrently` roda Back e Front no mesmo terminal: `npm run dev:all` (ou `dev:all:windows`).
- ✓ O front deve tolerar o back que demora a subir (verificação periódica de status).
- ✓ O proxy do Vite elimina o CORS **só em desenvolvimento**.
- ✓ Vercel + Render sempre precisam de CORS configurado no Spring.

Exercícios

1. Adicione `back`, `dev:all` e `dev:all:windows` ao seu projeto e rode com um comando só.

2. Configure o proxy e troque a `BASE_URL` para relativa; confirme na aba Network que as chamadas vão para a 5173.

3. Desligue o CORS do Spring e mostre que com proxy funciona e sem proxy não.

4. Crie as tarefas do VS Code para subir tudo pelo menu.

Para ir além

- npmjs.com/package/concurrently
- vite.dev/config/server-options#server-proxy

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 06?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web